



RS6x_7x mmWave Radar Development Quick Start Guide

Number of Pages:38

Document Version:1.4

Release Date:2026-08-31

Revision History

Version	Revision Description	Date
1.4	Renamed the motion point cloud to the presence point cloud; Corrected the IRQ DIP-switch description in UART-I ² C mode; Added a Q&A description of how to connect the radar to other embedded hardware for development; Added more detailed steps for running the first project;	2026-6-11
1.3	Added the method for porting and integrating HIF on the Host;	2025-12-17
1.2	Added a description of Adaptive Gain; Updated the method for downloading the CDK build tool;	2025-10-18
1.1	Updated the definition and illustration of the side-mounted vertical development-board orientation; Corrected the DUAL-UART DIP-switch description; Added an introduction to data output and visualization; Improved the Software Function Porting chapter;	2025-10-07
1.0	Initial release;	2025-09-15

Table of Contents

Revision History.....	ii
Table of Contents.....	iii
List of Figures.....	iv
List of Tables.....	v
1 Purpose and Scope.....	1
2 Overview.....	2
2.1 Development Kit.....	2
2.1.1 Hardware Development Kit.....	2
2.1.2 Software Development Kit.....	3
2.2 Related Documentation.....	3
3 Setting Up the Software Environment.....	5
3.1 Preparing the Build Toolchain.....	5
3.2 Installing Tools and Drivers.....	5
4 Running the First Project on the Development Board.....	6
4.1 Setting Up the Hardware Environment.....	6
4.2 Building.....	7
4.2.1 Directory Structure.....	7
4.2.2 Selecting the Board-Level Macro Configuration.....	8
4.2.3 Building.....	9
4.3 Burn.....	10
4.4 Running.....	11
5 Running the First Project on Target Hardware.....	14
5.1 Creating a Project Based on a Sample.....	14
5.2 Creating a Development Template for Target Hardware.....	14
5.3 Porting Communication Functions.....	17
5.3.1 Configuring Communication-Interface Functions.....	17
5.3.2 Typical HIF Application Scenarios.....	18
5.3.3 Configuring Communication Hardware I/O.....	19
5.4 Porting Radar Functions.....	19
5.5 Debug Interfaces.....	19
5.5.1 TRACE_IO.....	19
5.5.2 Logging System.....	20
5.5.3 PSIC Debug Protocol.....	21
5.6 Reliability Design.....	23
5.6.1 Watchdog.....	23
5.6.2 Task Monitor.....	23
5.6.3 Heartbeat Packet.....	23
5.6.4 Response Count.....	24
5.7 Embedded Resource Configuration and Verification.....	24
5.7.1 Configuring Code Resources.....	24
5.7.2 Verifying Embedded Resources.....	25
5.7.3 Measuring Computation Time.....	27
5.8 Porting and Verifying Low-Power Functions.....	28
5.8.1 Configuring Low-Power Mode.....	28
5.8.2 Verifying Power Consumption.....	28
5.9 Other SDK Modifications.....	28
6 Radar Debugging Methods.....	30
6.1 Generating Radar Configuration Interfaces.....	30
6.2 Testing and Verification.....	30
7 Q & A.....	31

List of Figures

Figure 1-1	Recommended Radar Evaluation, Design, and Development Workflow and Tools.....	1
Figure 2-1	mmWave Radar Modules and Development Boards.....	3
Figure 2-2	Downloading the CDK Tool.....	4
Figure 3- 1	setup.exe.....	5
Figure 3- 2	UART Device Information for the Development Kit.....	5
Figure 4-1	Hardware Switches.....	6
Figure 4-2	Switching Macro Configurations in the Project Configuration.....	8
Figure 4-3	IDE Interface.....	9
Figure 4- 4	Rebuild Interface.....	10
Figure 4-5	Connecting RadarDebugTool.....	11
Figure 4-6	Side-Mounted Vertical Installation of Development Boards.....	12
Figure 4-7	Starting the Radar from the Host.....	12
Figure 4-8	Displaying Point Clouds on the Host.....	13
Figure 4-9	Displaying Debug Information on the Host.....	13
Figure 4-10	Stopping Radar Data Reporting on the Host.....	13
Figure 5-1	Adding the myboard Folder.....	15
Figure 5-2	boards Folder Structure.....	15
Figure 5-3	board_config Macro Definition.....	16
Figure 5-4	Configuring MYBOARD.....	16
Figure 5-5	Modifying Related Macros in myboard_config.h.....	17
Figure 5-6	Configuring the Custom Template Global Macro.....	17
Figure 5-7	Modifying the UART PinMux.....	19
Figure 5-8	PSIC Debug Protocol.....	21
Figure 5-9	frame_info Display in the Tool.....	23
Figure 5-10	Link-File Path.....	25
Figure 5-11	Memory Distribution in the MAP File.....	25
Figure 5-12	Example Resource Statistics with CONFIG_DEBUG_SIZE_LEVEL = 1.....	26
Figure 5-13	Example Resource Statistics with CONFIG_DEBUG_SIZE_LEVEL = 2.....	26
Figure 5-14	Low-Power and CPU-Performance Configuration.....	28
Figure 7-1	Clean Project.....	31
Figure 7-2	Resistors Connecting Module Pins to Development-Board Hardware.....	32

List of Tables

Table 4-1	Communication-Mode Functions for Different Modules.....	7
Table 4-2	DIP-Switch Settings for Different Modes.....	7
Table 4-3	Tracking SoC Configurations.....	9
Table 4-4	Default Tracking Waveform Parameters.....	11
Table 5-1	Hardware Mapping Supported by shell/LOG/HIF.....	17
Table 5-2	Recommended Communication-Function Configurations.....	18
Table 5-3	Transfer Capabilities of Different Interfaces in Typical Scenarios.....	18
Table 5-4	Default CPU Frequencies.....	29

1 Purpose and Scope

This document describes in detail how to use the Possumic RS6x_7x Series mmWave Radar Development Kit. Through an explanation and porting of a typical example, it enables developers to quickly set up the software and hardware environments and run the first project on the target hardware.

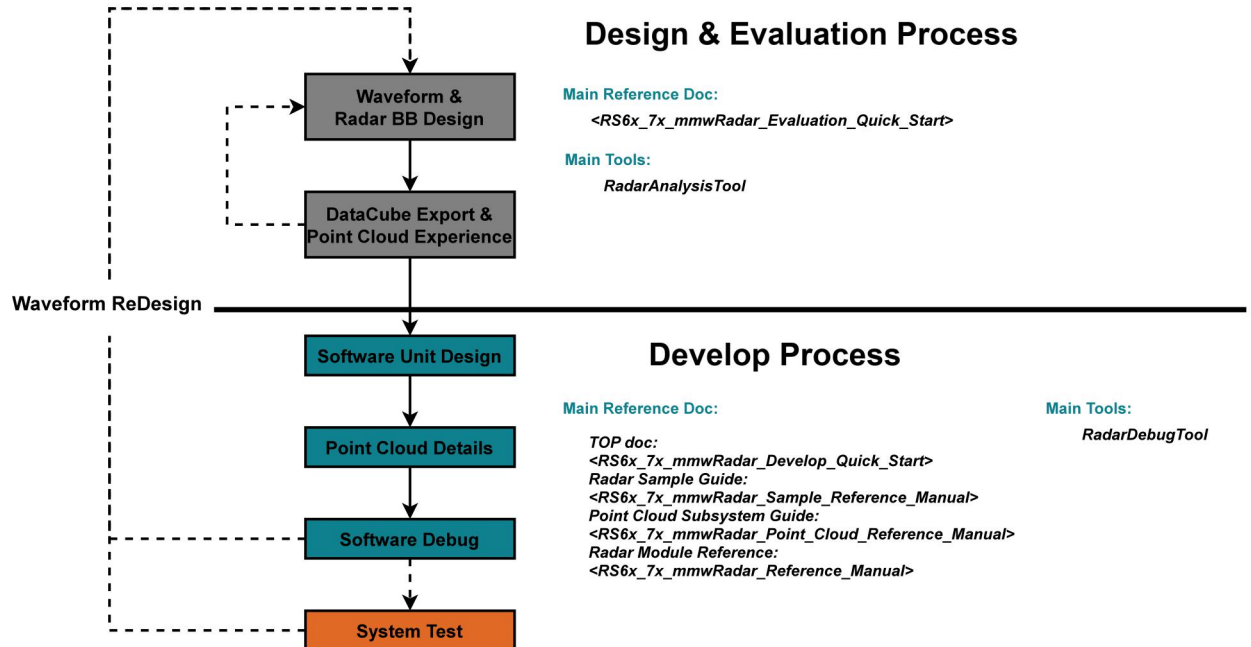


Figure 1-1 Recommended Radar Evaluation, Design, and Development Workflow and Tools

Developers are advised to use the SDK as shown in Figure 1-1. The SDK divides the development workflow into the design and evaluation phase and the development phase, and provides corresponding tools.

During the evaluation phase, RadarAnalysisTool and the RadarAnalysisTool firmware are primarily used for performance evaluation. Developers define the radar waveform by adjusting the parameter configurations provided by the tool, import waveform data or point clouds to the Host using the data acquisition function, and verify and optimize custom algorithms on the Host. The development phase can begin after the radar waveform and custom algorithms work together to achieve the expected results.

During the development phase, developers use the APIs and tools such as RadarDebugTool provided by the SDK, and refer to the Sample projects and SDK documentation, to deploy the radar waveform and custom algorithms selected during the evaluation phase to the embedded system, followed by testing, verification, and system optimization.

This document supports all devices in the RS6x and RS7x series.

2 Overview

This document is intended primarily for developers who are new to the RS6x_7x Series radar sensor devices. It demonstrates how to use the RS6x_7x Series development package to quickly set up the Tracking project on target hardware. Tracking includes configuration of the data communication link based on the HIF (Host Interface) application protocol, point-cloud processing, and point-cloud data reporting, and can serve as a developer's first introductory project.

This document first guides developers through setting up the software and hardware environments on the development board, running the Tracking project, and visualizing data with RadarDebugTool.

It then explains how to quickly deploy Tracking to target hardware by modifying key board-level configurations.

Finally, it introduces the debugging methods and tools provided by the SDK, which assist with data visualization, parameter tuning, unit testing, and algorithm optimization.

For information about using and configuring Samples, see the RS6x_7x mmWave Radar Sample Reference Manual.

For information about using the point-cloud processing reference flow provided by the SDK, see the RS6x_7x Point-Cloud Processing Reference Manual.

For information about using the radar driver and its features, see the RS6x_7x mmWave Radar Reference Manual.

For information about configuring and using the HIF communication protocol, see the RS6x_7x HIF Development Quick Start Guide.

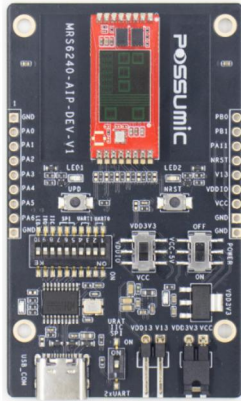
2.1 Development Kit

The development kit consists of a hardware development kit and a software development kit.

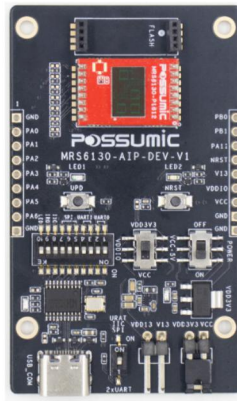
2.1.1 Hardware Development Kit

1. MRS6130_P1812/MRS6130_P1806/MRS6240_P2512/MRS6241_P2828_M62/MRS6241_P2840_M81: mmWave radar modules (red);
2. MRS6130-AIP-DEV-V1/MRS6240-AIP-DEV-V1/MRSx241-DEV-V1: development boards that carry the modules (black);
3. USB Type-C cable: Connects the PC to the development board.

CHIP: RS6240
MODULE: MRS6240-P2512
DEV BOARD: MRS6240-AIP-DEV-V1



CHIP: RS6130
MODULE: MRS6130-P1812
DEV BOARD: MRS6130-AIP-DEV-V1



CHIP: RS6130
MODULE: MRS6130-P1806
DEV BOARD: MRS6130-AIP-DEV-V1



CHIP: RS6241
MODULE: MRS6241-P2828-M62
DEV BOARD: MRSx241-DEV-V1



CHIP: RS6241
MODULE: MRS6241-P2840-M81
DEV BOARD: MRSx241-DEV-V1

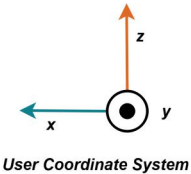
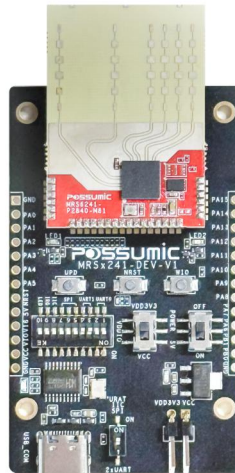


Figure 2-1 mmWave Radar Modules and Development Boards

2.1.2 Software Development Kit

The software development kit directory structure is as follows:

```
<RS6x_7x Series Development Package>/
├── Software_Kit          /* Primarily for software development */
│   ├── 01_Document      /* Software guides and manuals */
│   ├── 02_SDK            /* Software source package and build toolchain */
│   ├── 03_Tool           /* USB drivers, burn tools, radar GUI tools, and so on */
│   └── 04_Image          /* Precompiled demo firmware */
└── Hardware_Kit         /* Primarily for hardware development and evaluation */
    └── 01_Document      /* Hardware guides and manuals */
```

2.2 Related Documentation

- Build environment tools: Visit the Possumic website at <https://www.possumic.com/rs6130.html>

The screenshot shows the POSSUMIC website's Downloads page. The navigation menu includes Home, Products, Applications, Support, Company, Community, and Contact. The 'Downloads' tab is active. A sidebar on the left lists products (RS2111, RS6130, RS6240, RS6241, RS7241) and modules (MRS261L, MRS6130-P1806, MRS6240-P2512). The main content area shows a table of downloads with columns for Name, Related, Version, and Date. The file 'cdk-windows.zip' is highlighted with a red box.

Name	Related	Version	Date
+ RS6x_7x_mmWave_sdk.7z		V2.1.1	2026/08/14
+ cdk-windows.zip		V2.24.11	2025/10/16
+ MRS261L_Motion_Presence_EN.zip		R1.23.2	2026/05/17

Figure 2-2 Downloading the CDK Tool

As shown in Figure 2-2, click Product Package under Technical Resources. After registering an account, click to download cdk-windows.zip.

- Example projects:
 - ◆ mmWave project: [/Software Kit/02 SDK/psdf_sdk/project/mmwave/mmwave](#)
 - ◆ mmWave application project: [/Software Kit/02 SDK/psdf_sdk/project/mmwave/mmwave application](#)
- RadarDebugTool: [/Software Kit/03 Tool/RadarDebugTool](#)
- UART driver: [/Software Kit/03 Tool/USB Serial Driver](#)
- PC serial-port tool: To be provided by the developer.

3 Setting Up the Software Environment

Before using the SDK to build code, prepare the build environment and related software.

3.1 Preparing the Build Toolchain

1. Extract the CDK package, enter the directory, and double-click setup.exe.

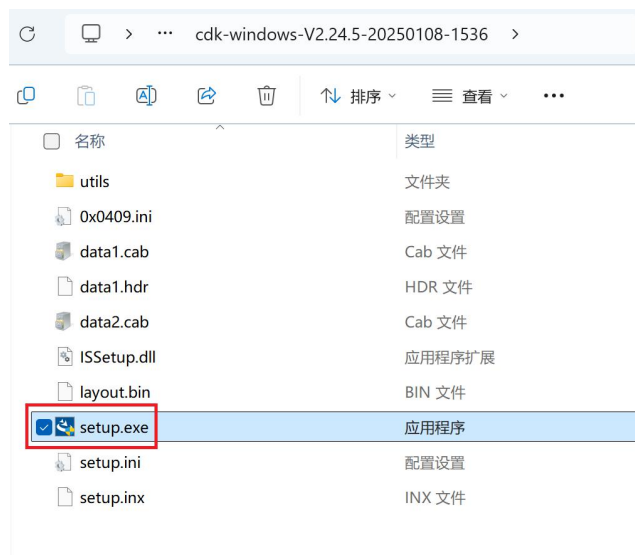


Figure 3-1 setup.exe

2. Complete the installation.

3.2 Installing Tools and Drivers

The baseboard of the development kit integrates USB-to-UART hardware. Install the driver to use the UART ports.

Driver path: [/Software Kit/03 Tool/USB Serial Driver](#)

After installation, connect the development kit to the PC. One UART port is displayed in UART-SPI/UART-I²C mode, or two UART ports are displayed in DUAL-UART mode.

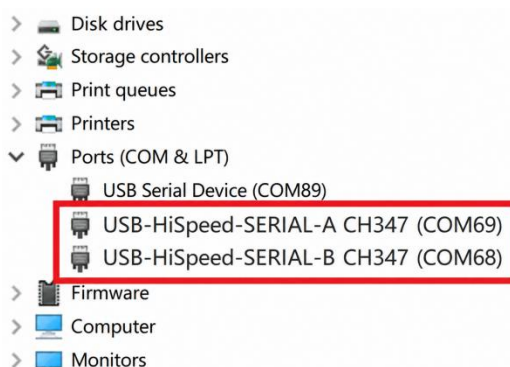


Figure 3-2 UART Device Information for the Development Kit

4 Running the First Project on the Development Board

Using the Tracking project as an example, this chapter describes the code structure, how to build the project source code, burn the firmware to the development board, and configure and visualize data in RadarDebugTool. Tracking demonstrates three important functions: an HIF-based configuration link, the point-cloud digital signal processing flow, and point-cloud data visualization using RadarDebugTool.

The Tracking project is located at [/Software Kit/02 SDK/psdf sdk/project/mmwave/mmwave/Tracking.](#)

4.1 Setting Up the Hardware Environment

Each development board has two groups of DIP switches with identical functions, indicated by orange and green boxes. The group in the orange box is FUNC SEL, and the group in the green box is MODE SEL. A single-pole double-throw switch labeled POWER and a tactile switch labeled UPD (UPDATE) are also commonly used during development.

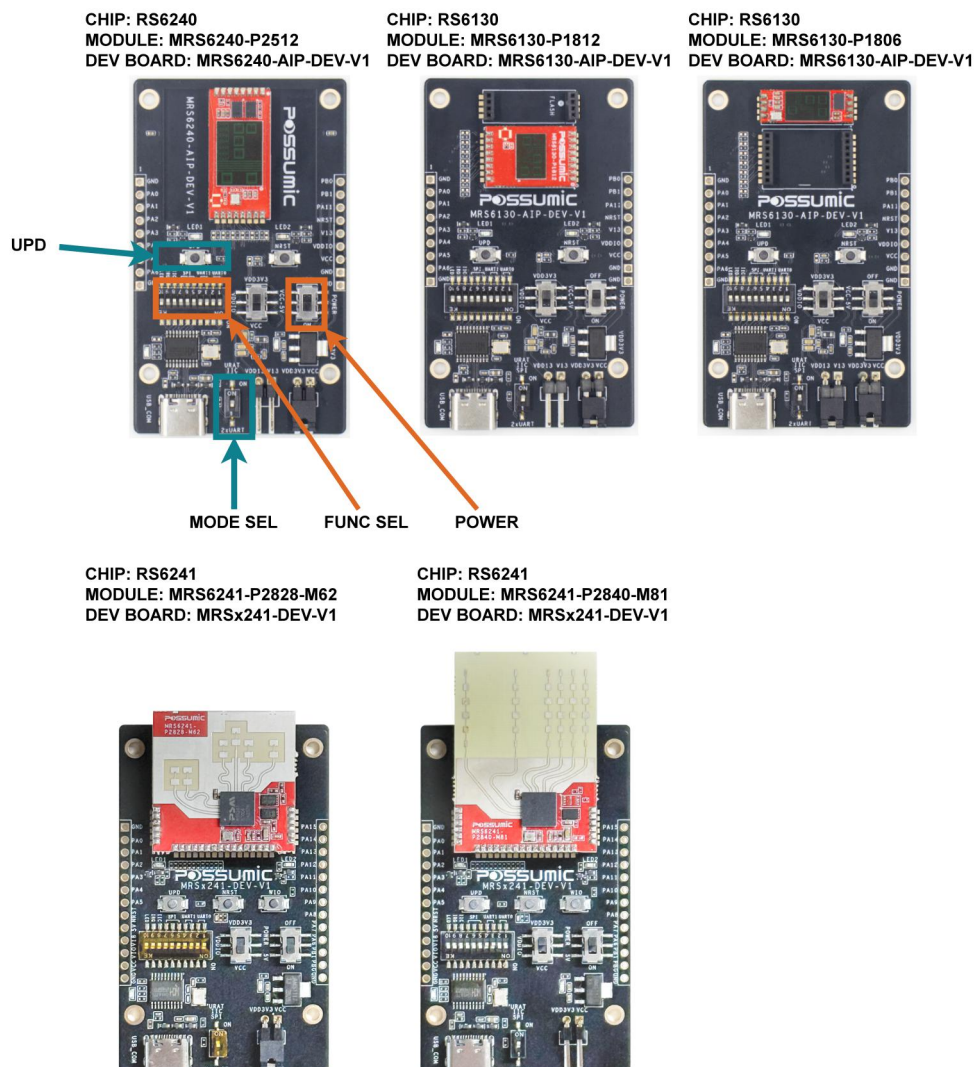


Figure 4-1 Hardware Switches

The MRS6240-AIP-DEV-V1, MRS6130-AIP-DEV-V1, and MRSx241-DEV-V1 development boards support three communication modes: DUAL-UART, UART-SPI, and UART-I²C. The modes are controlled by the FUNC SEL and MODE SEL DIP-switch groups.

Table 4-1 Communication-Mode Functions for Different Modules

Mode Selection	DUAL-UART			UART-SPI/UART-IIC		
Hardware Interface	UART0	UART1	SPI	UART0	UART1	SPI/IIC
MRS6130-P1806	Tx/Rx	Tx ONLY	-	Tx/Rx	-	NOT SUPPORT
MRS6130-P1812	Tx/Rx	Tx/Rx	-	Tx/Rx	-	Tx/Rx
MRS6240-P2512	Tx/Rx	Tx/Rx	-	Tx/Rx	-	Tx/Rx
MRS6241-P2828	Tx/Rx	Tx/Rx	-	Tx/Rx	-	Tx/Rx
MRS6241-P2840	Tx/Rx	Tx/Rx	-	Tx/Rx	-	Tx/Rx

The DIP-switch positions for each mode are listed in the following table:

Table 4-2 DIP-Switch Settings for Different Modes

	FUNC SEL										MODE SEL
Switch Position	1	2	3	4	5	6	7	8	9	10	1
	UART0	UART0	UART1	UART1	SPI	SPI	SPI	IIC	IRQ	LED	2xUART
DUAL-UART	ON	ON	ON	ON	OFF	OFF	OFF	OFF	OFF	ANY	OFF
UART-SPI	ON	ON	OFF	OFF	ON	ON	ON	OFF	ON	ANY	ON
UART-IIC	ON	ON	OFF	OFF	OFF	OFF	OFF	ON	ON	ANY	ON

In the preceding table, ON means that the DIP switch is set toward the position marked ON.

SPI supports a transfer rate of up to 60 Mbps (depending on CPU frequency), which supports transmission of a 256 KB DataCube at approximately 10 Hz. UART supports up to 2 Mbps. Due to limitations of some PC hardware, the 2 Mbps baud rate may be unstable. If bit errors occur at 2 Mbps, use 1 Mbps.

Default Tracking reference-project configuration: **The MRS6130-P1806 module uses DUAL-UART mode. UART0 runs at 115200 baud for firmware download and printing, and UART1 runs at 1 Mbps for data reporting. The MRS6130-P1812, MRS6240-P2512, MRS6241-P2828, and MRS6241-P2840 modules use UART-SPI mode with a UART baud rate of 115200.**

This document uses the MRS6240-P2512 as an example. If the MRS6130-P1806 module is used, subsequent SPI operations correspond to DUAL-UART operations.

Note: After changing the MODE SEL DIP switch, power-cycle the module by toggling the POWER switch for the mode change to take effect.

For more information about the development-board hardware, see the user manual for the corresponding module development board.

4.2 Building

Before proceeding, ensure that the software environment has been set up according to Chapter 3.

4.2.1 Directory Structure

The SDK source directory structure is shown below. Locate the Tracking project directory.

```
<home>/Software_Kit/02_SDK/
psdf_sdk          /* Software code */
|__ kernel        /* RTOS source and OS abstraction layer */
|__ libs          /* Static libraries (.a) and binary files (.img) */
```

__ platform	<u><i>/* Device-platform drivers and configuration files */</i></u>
__ project	<u><i>/* Application projects */</i></u>
__mmwave	<u><i>/* Radar-related projects */</i></u>
__ common	<u><i>/* Common files for example projects */</i></u>
__ radar_analysis	<u><i>/* Radar-analysis project used with RadarAnalysisTool */</i></u>
__mmwave	<u><i>/* DataCube reporting projects */</i></u>
__ReportDataCube1D	<u><i>/* 1D-frame DataCube reporting project */</i></u>
__ReportDataCube2D	<u><i>/* 2D-frame DataCube reporting project */</i></u>
__mmwave_application	
__Posture	<u><i>/* Posture point-cloud example project */</i></u>
__Sleep	<u><i>/* Sleep-detection example project */</i></u>
__Tracking	<u><i>/* Target-tracking example project */</i></u>
__subsys	<u><i>/* Subsystem projects */</i></u>
__subsys	<u><i>/* Functionally independent subsystems */</i></u>
__hif	<u><i>/* Host Interface subsystem */</i></u>
__mmw	<u><i>/* Radar subsystem */</i></u>
__mmw_algorithm	<u><i>/* Basic radar algorithms */</i></u>
__mmw_application	<u><i>/* Radar application algorithms, including motion& presence point clouds */</i></u>
__tools	<u><i>/* Tool scripts */</i></u>

4.2.2 Selecting the Board-Level Macro Configuration

Tracking selects the development board through a predefined macro in the project configuration. Different boards initialize different I/Os and peripherals, select different CPU series, and configure different CPU frequencies. Currently, the RS61xx Series can use only CPUS, while the RS62xx Series can use only CPUF, which supports DSP acceleration and a higher clock frequency.

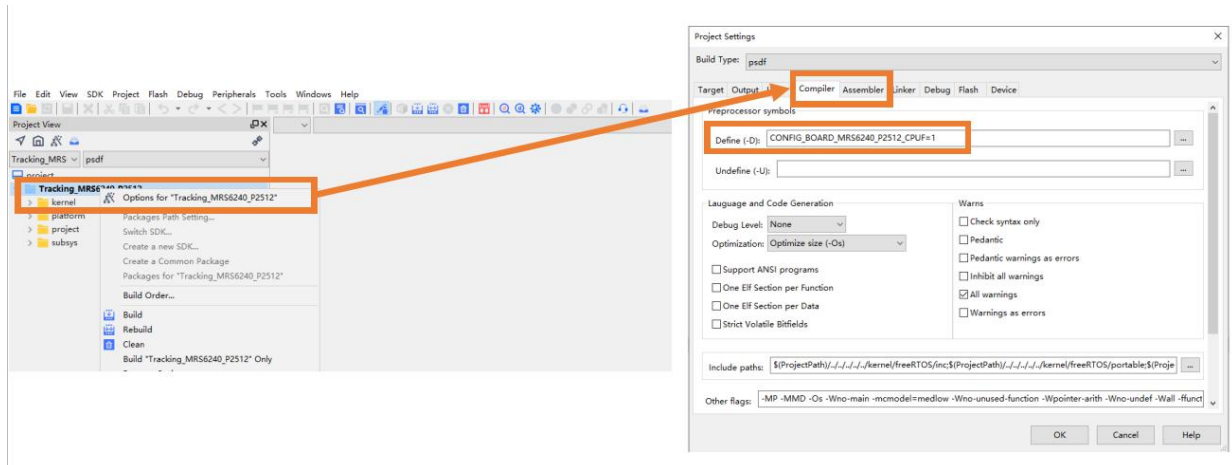


Figure 4-2 Switching Macro Configurations in the Project Configuration

Ensure that the correct board-level macro definition is selected on both the Compiler and Assembler tabs. The cdk_6240_cpuf project selects CONFIG_BOARD_MRS6240_P2512_CPUF=1 by default. The cdk_6130_1812_cpus project selects CONFIG_BOARD_MRS6130_P1812=1 by default. The cdk_6130_1806_cpus project selects CONFIG_BOARD_MRS6130_P1806=1 by default. The cdk_6241_2828_xip_cpuf project selects CONFIG_BOARD_MRS6241_P2828_M62_CPUF=1 by default. The cdk_6241_2840_xip_cpuf project selects CONFIG_BOARD_MRS6241_P2840_M81_CPUF=1 by default. The SDK currently includes five supported board-level configurations:

- CONFIG_BOARD_MRS6130_P1812
- CONFIG_BOARD_MRS6130_P1806
- CONFIG_BOARD_MRS6240_P2512_CPUF
- CONFIG_BOARD_MRS6241_P2828_M62_CPUF
- CONFIG_BOARD_MRS6241_P2840_M81_CPUF

For available board-level macro definitions, see the configuration header files in [/Software Kit/02 SDK/psdf sdk/platform/boards/](#). These files are included by the common configuration file common.h, located in [/Software Kit/02 SDK/psdf sdk/platform/utis](#).

Table 4-3 Tracking SoC Configurations

BOARD	CPU Frequency	Communication Mode	Print Output Interface	Data Output Interface	Motion Point-Cloud Output
MRS6130-P1806	128 MHz	DUAL-UART	UART1@115200	UART0 @1Mbps	Supported
MRS6130-P1812	128 MHz	UART-SPI	UART0@115200	SPI	Supported
MRS6240-P2512	256 MHz	UART-SPI	UART0@115200	SPI	Supported
MRS6241-P2828	256 MHz	UART-SPI	UART0@115200	SPI	Supported
MRS6241-P2840	256 MHz	UART-SPI	UART0@115200	SPI	Supported

4.2.3 Building

1. Enter [/Software Kit/02 SDK/psdf sdk/project/mmwave/mmwave_application/Tracking](#), and then enter the project directory corresponding to the device model in use.
2. Double-click project.cdkproj to open the project, as shown in Figure 4-3.

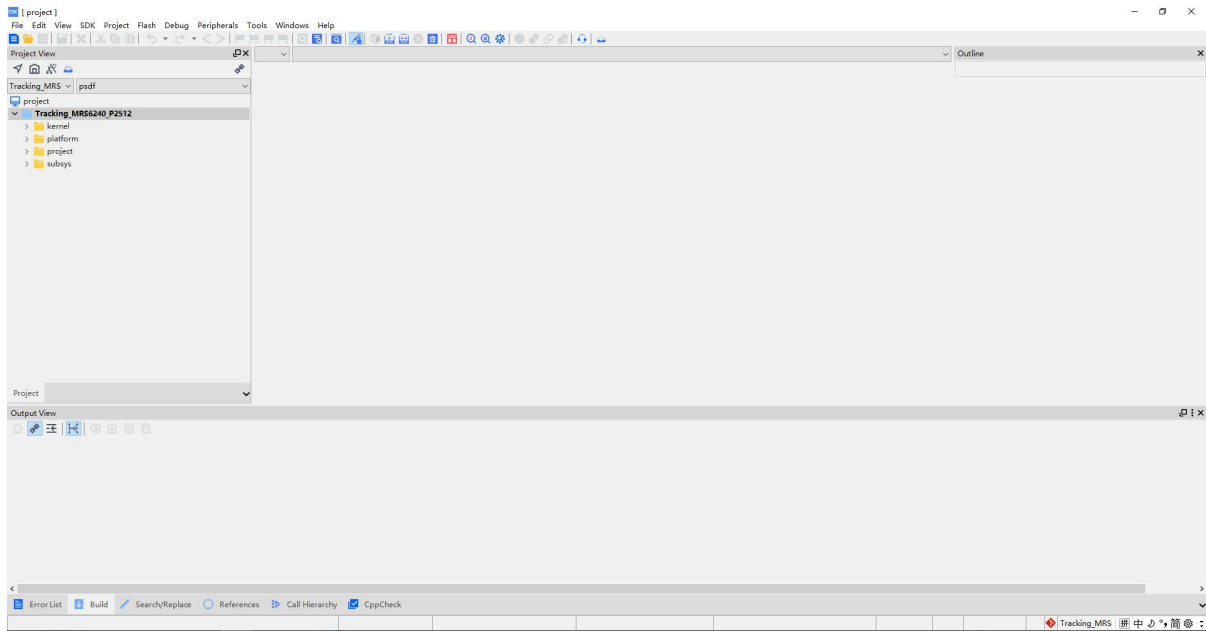


Figure 4-3 IDE Interface

3. Click Rebuild Project on the IDE toolbar to rebuild the project, as shown in Figure 4-4. Check the build output and IMG packaging output. The generated data is located in the output folder under the project-file (.cdkproj) directory.-

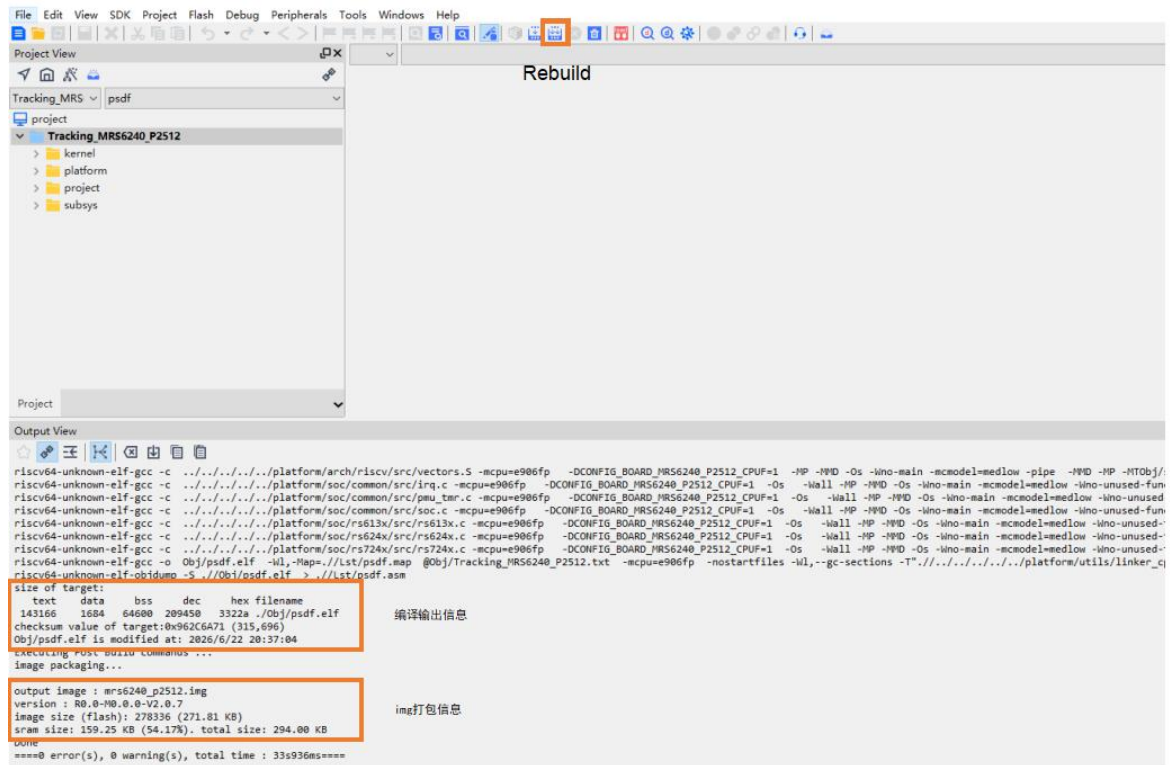


Figure 4-4 Rebuild Interface

Note: For detailed guidance on project configuration and the IDE, see the '[RS6x_7x_SDK_Reference_Manual](#)'.

4.3 Burn

Burn is the process of loading the compiled IMG file into the development kit's SPI Flash using RadarDebugTool.

1. Set the development kit to burn mode.

- (1) According to Table 4-2, set the switches to UART-SPI mode. For the MRS6130-P1806 module, set them to DUAL-UART mode.
- (2) Set the POWER switch to OFF. The power LED turns off.
- (3) Press and hold the UPD button, and then set the POWER switch to ON. The power LED turns on.
- (4) Continue holding the UPD button for approximately 2 seconds, and then release it.

2. Connect the burn tool and burn the firmware:

- (1) For UART-SPI mode, set Communication Method to SPI, select the SPI port, and set the SPI frequency to 60 MHz.
- (2) For the MRS6130-P1806, set Communication Method to UART, select the port with the larger COM number in most cases, and set the baud rate to 115200.
- (3) Click Connect to connect to the radar. Then click Download and select the firmware to download. The burn starts automatically. The burn succeeds when the progress bar reaches 100%.
- (4) After the burn is complete, press NRST to reset the radar so that it starts operating with the new firmware.

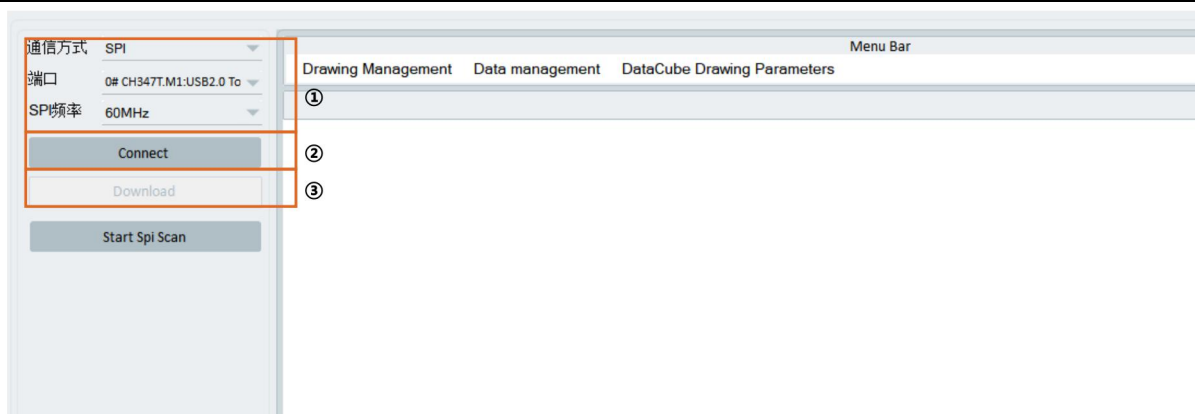


Figure 4-5 Connecting RadarDebugTool

If the connection fails, the serial-port number may be incorrect or the development kit may not be in burn mode. Repeat Step 1 (set the development kit to burn mode), and use Windows Device Manager to confirm the serial-port number or select a different port.

If the burn fails, the UART baud rate may be too high. Reduce the baud rate and try again.

4.4 Running

The Tracking example primarily demonstrates the HIF communication link and point-cloud algorithms. Because interface transfer rates vary by module type, the Tracking project configures different default waveform parameters.

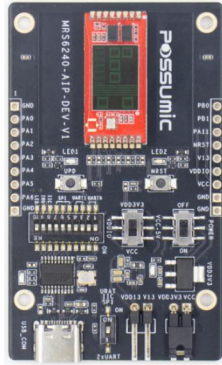
Table 4-4 Default Tracking Waveform Parameters

BOARD	Range Resolution	Maximum Detection Range	Velocity Resolution	Maximum Unambiguous Velocity	Frame Period
MRS6130-P1806	0.08m	20.48m	0.2m/s	3.2m/s	100 ms
MRS6130-P1812	0.08m	20.48m	0.2m/s	3.2m/s	50 ms
MRS6240-P2512	0.08m	20.48m	0.2m/s	3.2m/s	50 ms
MRS6241-P2828	0.08m	20.48m	0.2m/s	3.2m/s	50 ms
MRS6241-P2840	0.08m	20.48m	0.2m/s	3.2m/s	50 ms

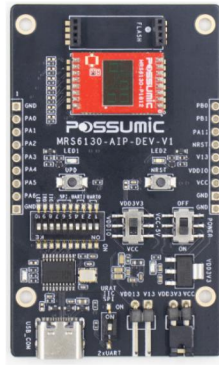
Based on the device type, confirm the communication mode in the Communication Mode column of Table 4-3, and verify that the DIP switches are in the correct positions according to Table 4-2.

Tracking supports a vertically oriented side-mounted radar. The side-mounted vertical orientations of different development boards are shown in Figure 4-6.

CHIP: RS6240
MODULE: MRS6240-P2512
DEV BOARD: MRS6240-AIP-DEV-V1



CHIP: RS6130
MODULE: MRS6130-P1812
DEV BOARD: MRS6130-AIP-DEV-V1



CHIP: RS6130
MODULE: MRS6130-P1806
DEV BOARD: MRS6130-AIP-DEV-V1



CHIP: RS6241
MODULE: MRS6241-P2828-M62
DEV BOARD: MRSx241-DEV-V1



CHIP: RS6241
MODULE: MRS6241-P2840-M81
DEV BOARD: MRSx241-DEV-V1

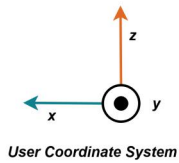
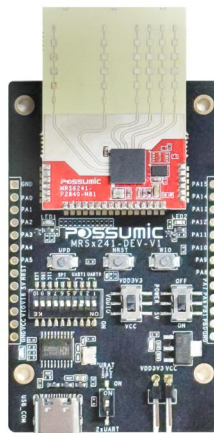


Figure 4-6 Side-Mounted Vertical Installation of Development Boards

Perform the following steps:

1. Use the Host button to start radar data reporting. For the MRS6130-P1806, select UART as the communication method, select the COM port corresponding to UART0, which is generally the port with the larger COM number, and set the baud rate to 1 Mbps.

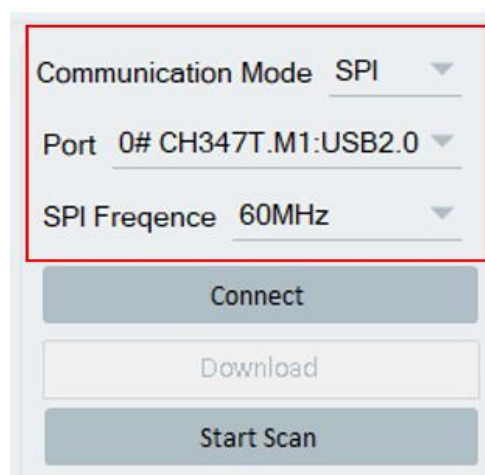


Figure 4-7 Starting the Radar from the Host

- Select the port connected to the PC, and click Connect to connect to the radar.
- Click Download to burn the firmware. Skip this step if firmware has already been burned to the radar.
- Click Start Scan to start data reception on the Host.
- For the MRS6130-P1806, after the burn is complete, switch the port to the UART0 serial port to receive data.

2. In RadarDebugTool, select the point cloud, DataCube information, or debug information to display. Right-click the plot window, select the desired three-view data, and set the plot type to Scatter Plot.

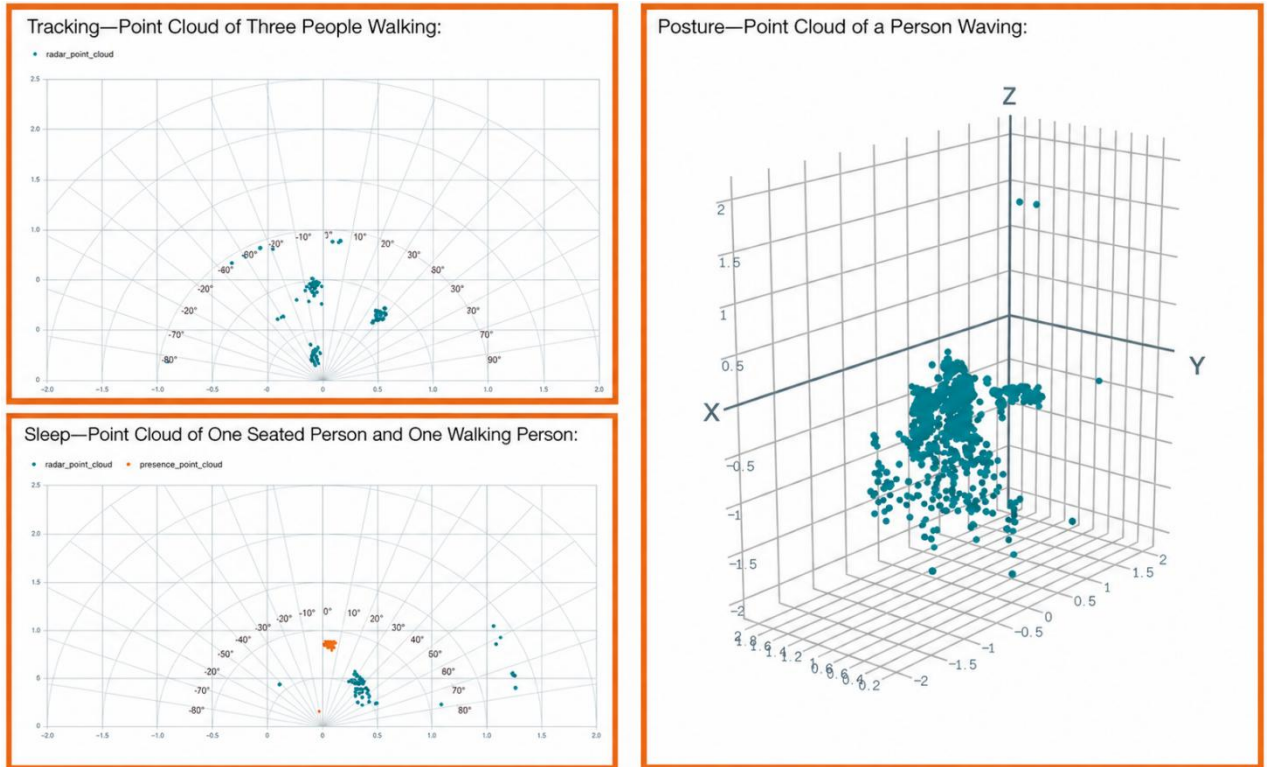


Figure 4-8 Displaying Point Clouds on the Host

Figure 4-8 shows the point-cloud results of three Samples in typical application scenarios. The two views on the left are 2D X-Y views, and the view on the right is a 3D point-cloud view. For more information about RadarDebugTool, see the RS6x_7x RadarDebugTool User Manual.

3. Developers can switch to the Setting screen at any time, select a waveform, and click One-Click Configuration to configure the radar waveform and observe the point cloud. The Radar Antenna Operating Mode setting differs between RS6130 and RS6240. RS6130 currently supports only 1T3R, while RS6240 supports only 2T4R. Reported data is displayed in the red box in Figure 4-9.

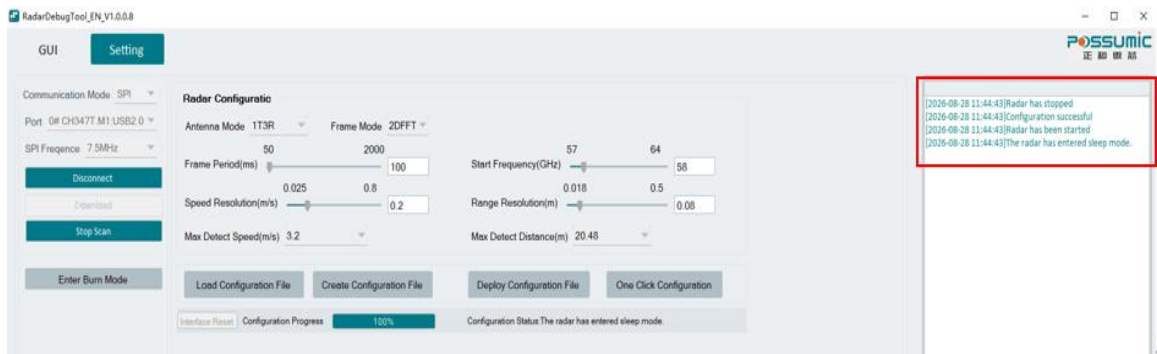


Figure 4-9 Displaying Debug Information on the Host

4. Click Stop Scan on the Host to stop receiving radar data.

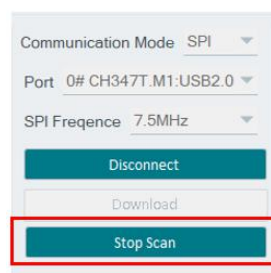


Figure 4-10 Stopping Radar Data Reporting on the Host

5 Running the First Project on Target Hardware

This chapter describes how to port the Tracking project to target hardware. The primary challenge is modifying and adapting the data-transfer interfaces.

HIF (Host Interface) is the standard data structure provided by the SDK and resides at the application layer of the communication model. All tools provided by the SDK support HIF, and Host-side sample code is provided to simplify HIF implementation on the Host. Developers are strongly advised to use HIF directly for data transfer. For more information, see the RS6x_7x HIF Development Quick Start Guide.

Point-cloud digital signal processing is independent of the hardware platform, so the point-cloud processing algorithms do not need to be ported. For implementation details, see the RS6x_7x mmWave Radar Point-Cloud Processing Reference Manual.

The target-hardware design must comply with specific constraints. For these constraints, see the RS6xxx Hardware Design and PCB Layout Guide.

5.1 Creating a Project Based on a Sample

First, copy the Tracking project and rename it to a new directory.

Note that digital signal processing algorithms provided through SDK interfaces or open-source code may be adjusted slightly as the SDK is updated, which can change the point-cloud output characteristics. If the SDK-provided digital signal processing algorithms are customized, separate the modified portions to decouple the product algorithms from the SDK.

For each SDK update, review the Release Notes, determine whether bugs in existing interfaces have been fixed, and evaluate how changes in point-cloud output characteristics affect the algorithms.

For more information about creating projects for the RS6130, RS6240, and RS6241 Series devices, see the ['RS6x_7x_SDK_Reference_Manual'](#).

5.2 Creating a Development Template for Target Hardware

The SDK uses development templates to configure device PINMUX settings and functional modules. Create a custom development template for the target hardware. Default templates are located in platform/boards. Assuming the target board is named myboard, create a template as follows.

1. Enter platform/boards. Copy the mrs6240_p2512 folder for the RS6240 Series, mrs6130_p1812 for the RS6130 Series, mrs6241_p2828 for MRS6241_P2828, or mrs6241_p2840 for MRS6241_P2840, and rename it myboard. In myboard, rename mrs6240_p2512.c and mrs6240_p2512_config.h to myboard.c and myboard_config.h. Add the myboard folder to the project using Add Source Folder.

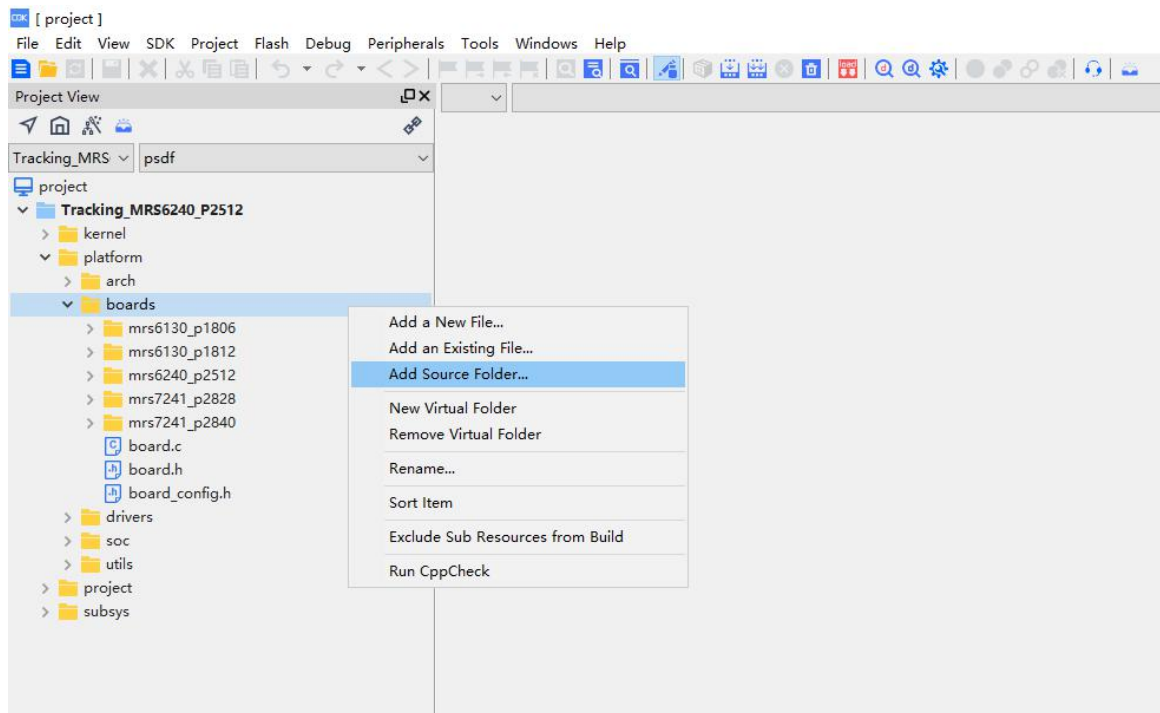


Figure 5-1 Adding the myboard Folder

2. The boards directory now appears as shown below:

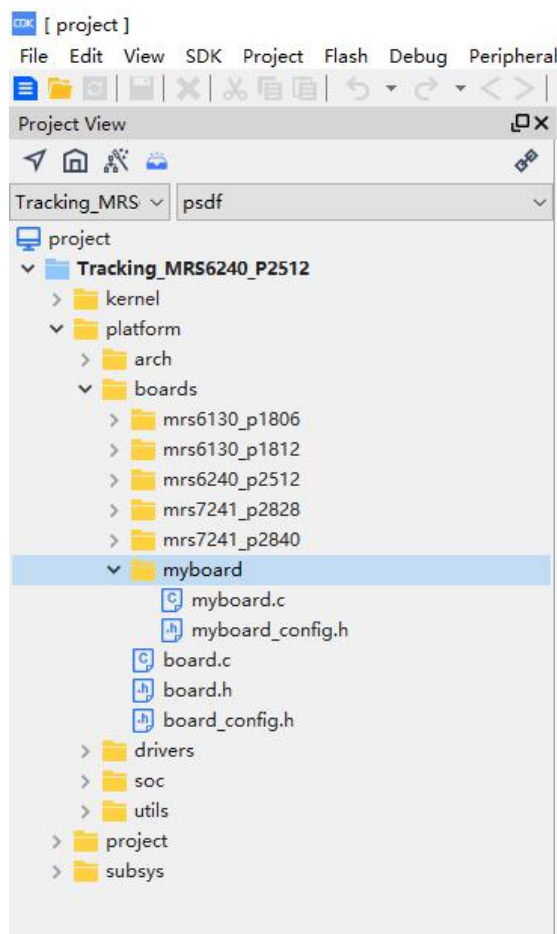


Figure 5-2 boards Folder Structure

3. Open board_config.h and add the macro and header-file inclusion:

```

#ifndef CONFIG_BOARD_MRS6242_P2512_CPUS
#define CONFIG_BOARD_MRS6242_P2512_CPUS ..... 0
#endif

#ifndef CONFIG_BOARD_MRS6242_P2512_CPUPF
#define CONFIG_BOARD_MRS6242_P2512_CPUPF ..... 0
#endif

#ifndef CONFIG_BOARD_MYBOARD
#define CONFIG_BOARD_MYBOARD ..... 0
#endif

/* Private includes.
 *
 */

#if CONFIG_BOARD_MRS6130_P1806
#include "mrs6130_p1806/mrs6130_p1806_config.h"
#elif CONFIG_BOARD_MRS6130_P1812
#include "mrs6130_p1812/mrs6130_p1812_config.h"
#elif CONFIG_BOARD_MYBOARD
#include "myboard/myboard_config.h"
#elif (CONFIG_BOARD_MRS6240_P2512_CPUS || CONFIG_BOARD_MRS6240_P2512_CPUPF)
#include "mrs6240_p2512/mrs6240_p2512_config.h"
#elif ((CONFIG_BOARD_MRS6242_P2512_CPUS) || (CONFIG_BOARD_MRS6242_P2512_CPUPF))
#include "mrs6242_p2512/mrs6242_p2512_config.h"
#else
#error "No board has been selected!!!"
#endif

```

Figure 5-3 board_config Macro Definition

4. Open myboard.c and modify the switch and log configuration macros:

```

#include "soc_config.h"

#if (CONFIG_BOARD_MYBOARD)

#include "hal_types.h"
#include "hal_dev.h"
#include "hal_board.h"

#include "hal_power.h"
#include "hal_clock.h"
#include "hal_gpio.h"
#include "hal_uart.h"
#include "hal_spi.h"
#include "hal_i2c.h"
#include "hal_flash.h"
#include "ll_rcc_dev.h"
#include "ll_utils.h"
#include "log_port.h"

#if CONFIG_SHELL
#include "shell.h"
#endif

#ifndef CONFIG_BOARD_LOG_LEVEL
#define CONFIG_BOARD_LOG_LEVEL ..... LEVEL_ERR
#endif

#define LOG_MODULE ..... "myboard"
#define LOG_LEVEL ..... CONFIG_BOARD_LOG_LEVEL
#include "log.h"

```

Figure 5-4 Configuring MYBOARD

5. Open myboard_config.h and modify the macros related to the custom template:

```

#ifndef _MYBOARD_CONFIG_H
#define _MYBOARD_CONFIG_H

/* Includes.
 *
 */

/* Exported macro.
 *
 */

#if CONFIG_BOARD_MYBOARD
#define CONFIG_BOARD_STRING ..... "myboard"
#define CONFIG_SOC_RS6130 ..... 1

#ifndef CONFIG_HEAP_SIZE
#define CONFIG_HEAP_SIZE ..... (1024 * 40)
#endif

#ifndef CONFIG_KERNEL
#define CONFIG_KERNEL ..... 1
#endif

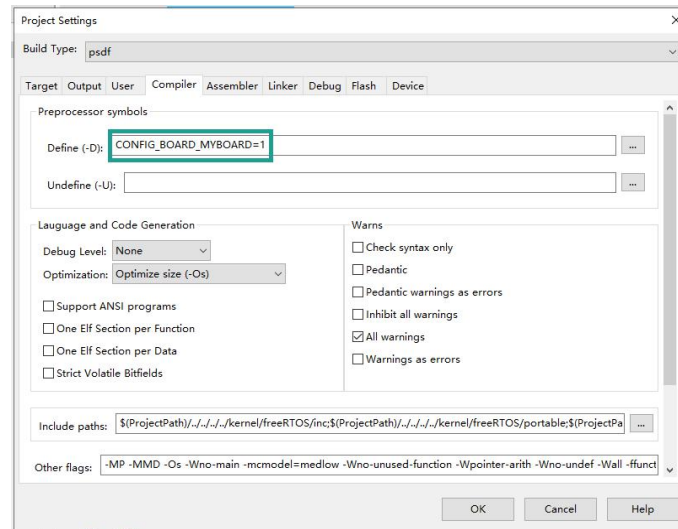
#endif /* CONFIG_BOARD_MYBOARD */

#endif /* _MYBOARD_CONFIG_H */

```

Figure 5-5 Modifying Related Macros in myboard_config.h

6. In the project build configuration, select Compiler. For build-option configuration, see the '[RS6x_7x_SDK_Reference_Manual](#)'. In the Compiler and Assembler global macro-definition boxes shown in Figure 5-6, set the custom template configuration macro CONFIG_BOARD_MYBOARD to 1.

**Figure 5-6 Configuring the Custom Template Global Macro**

For more information about creating development templates, see the '[RS6x_7x_SDK_Reference_Manual](#)'.

5.3 Porting Communication Functions

Communication-function porting must comply with the SDK design guidelines. This chapter describes how to configure communication by using a development template. For configuration of other peripherals and the CPU, see the '[RS6x_7x_SDK_Reference_Manual](#)' and the corresponding demos.

5.3.1 Configuring Communication-Interface Functions

Developers are strongly advised to use the shell, LOG, and HIF communication protocols provided by the SDK instead of communicating through low-level drivers. These application-layer protocols can be mapped to different hardware by modifying the development-template settings described in Section 5.2.

Table 5-1 Hardware Mapping Supported by shell/LOG/HIF

	UART0 & UART1	SPI(Slave)	IIC
shell	√	×	×
LOG	√	×	×
HIF	√	√	√

The SDK imposes the following restrictions on communication interfaces:

1. shell and LOG can be mapped only to UART.
2. HIF and shell/LOG cannot be mapped to the same UART hardware.
3. shell and LOG can be mapped to the same UART hardware.
4. HIF can be mapped to only one hardware interface (UART0/UART1/SPI/I²C) and cannot be used on multiple interfaces.
5. The device supports firmware burn only through UART0, I²C, or SPI.
6. If shell, LOG, or HIF is mapped to a hardware interface, do not operate that hardware through the low-level interface. For example, if shell is mapped to UART0, do not use the UART driver API to operate

UART0; otherwise, shell may not work correctly. To implement a custom protocol using a low-level driver, do not map shell, LOG, or HIF to that hardware.

In summary, select one of the combinations listed in Table 5-2 to configure communication functions, and use HIF for data transfer instead of the low-level UART driver.

Table 5-2 Recommended Communication-Function Configurations

	UART0	UART1	SPI(Slave) or IIC
Dual-UART Configuration 1	shell/LOG	HIF	DISABLED
Dual-UART Configuration 2	HIF	shell/LOG	DISABLED
UART + SPI/I ² C Configuration 1	shell/LOG	DISABLED	HIF
No-UART Configuration	DISABLED	DISABLED	HIF
Single-UART Configuration	HIF	DISABLED	DISABLED

Note 1: The No-UART Configuration supports firmware update only through SPI/I²C. All other configurations support firmware update through UART0 or SPI.

Note 2: Available SPI frequencies depend on the CPU frequency. For supported frequencies, see the '[RS6x_7x_SDK_Reference_Manual](#)'.

For more information about shell/LOG/HIF and their configuration, see the '[RS6x_7x_SDK_Reference_Manual](#)'.

Note 3: The MRS6130-P1806 supports only Dual-UART Configuration 2 and Single-UART Configuration.

To receive HIF messages on the Host or learn more about HIF porting, see the RS6x_7x HIF Reference Manual for HIF communication methods and sample code.

For details about porting LOG, see Section 5.5.2.

5.3.2 Typical HIF Application Scenarios

To ensure data-transfer efficiency, use HIF for data communication. HIF supports different target transfer rates when mapped to different hardware interfaces, allowing developers to select an interface based on application requirements. Table 5-3 lists the transfer time required by different HIF hardware mappings in typical applications. Typical interface applications and data sizes are as follows:

1. 2D DataCube reporting: The range FFT length is 256, the Doppler FFT length is 32, and the MIMO configuration is 2T4R. Each frame reports 2D DataCube data from all current antennas, for a total of 256 KB.
2. 1D DataCube reporting: The range FFT length is 256, the MIMO configuration is 2T4R, and the radar uses a 1D-FFT processing flow. Each frame reports the 1D-FFT spectra of all antennas, for a total of 8 KB.
3. PointCloud: Reporting one valid point-cloud target requires 12 bytes.

Table 5-3 Transfer Capabilities of Different Interfaces in Typical Scenarios

	DataCube 256 × 32, 2T4R, Uncompressed, 256 KB Data	DataCube 256x64, 2T4R 2 × Compression, 512 KB Data	DataCube-1D 256 × 1, 2T4R, 8 KB Data	PointCloud 512 Points
SPI @ 60 Mbps (Development Board, Single-Line SPI)	80 ms	160 ms	2.5 ms	2 ms
SPI@60M bps (QUAD-SPI)	30 ms	40 ms	0.625 ms	0.5 ms

UART@1M baud rate	3000 ms	6000 ms	95 ms	50 ms
UART@2M baud rate	1500 ms	3000 ms	47.5 ms	25 ms

Note that the RS6x_7x Series supports Quad SPI. For higher frame rates, developers can select an interface device that supports Quad SPI to report DataCube data at a higher rate. To use Quad SPI, configure SPI as Quad SPI in the SDK development template.

When using UART to connect to a PC for data reporting, verify that the PC USB interface supports a 2 Mbps UART baud rate to prevent reporting failures.

5.3.3 Configuring Communication Hardware I/O

UART1 can be configured on PA0/PA1 (default) or PA8/PA9. SPI supports only one PinMux configuration. Based on the hardware schematic, modify the PINMUX pins in the board-level configuration file `myboard.c`, as shown in Figure 5-7. For details, see the '[RS6x_7x_SDK_Reference_Manual](#)'.

```

DEFINE_PINMUX_WEAK(uart1, 2,
{
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOA_P0_F3_UART1_TX, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_A, GPIO_PIN_1, {GPIOA_P1_F3_UART1_RX, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
});

DEFINE_PINMUX_WEAK(uart1_deinit, 2,
{
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOX_Pn_F15_DIS, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_A, GPIO_PIN_1, {GPIOX_Pn_F15_DIS, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
});

```

Figure 5-7 Modifying the UART PinMux

5.4 Porting Radar Functions

See the RS6x_7x mmWave Radar Sample Reference Manual for information about quickly porting mmWave radar software functions based on the radar framework.

5.5 Debug Interfaces

The SDK provides three debugging methods to meet timing, logging, and complex data-transfer debugging requirements.

5.5.1 TRACE_IO

TRACE_IO provides a set of APIs that apply different pulse waveforms to I/Os for measuring code execution speed and debugging timing. A logic analyzer or oscilloscope is required to observe the pin voltage. Define `CONFIG_LOG_IO_TAG_MASK` and `CONFIG_LOG_IO_MASK` as follows:

1. `CONFIG_LOG_IO_TAG_MASK` serves as the TRACE_IO tag. Its value only needs to be representable by `uint32_t`; for example, it can be set to 0x1000.
2. `CONFIG_LOG_IO_MASK` specifies the pins to use. Bits 0 through 11 indicate whether PA0 through PA11 are used. Set the corresponding bit to 1 for each TRACE_IO pin. For example, to use PA0, set the macro to 0B000000000001 (0x01).

When selecting TRACE_IO, note the following:

- Only PA pins can be selected for TRACE_IO. PB0-group drivers are not currently supported.
- Before selecting a TRACE_IO pin, determine whether the pin is already used by another function. For example, PA6 is used by IRQ in UART-SPI/UART-I²C mode. Using PA6 for TRACE_IO in this mode may cause abnormal operation.
- When using the MRS6130-P1806 module, only PA0 can be used for TRACE_IO. Because PA0 is used by UART1 by default, UART1 cannot be used when TRACE_IO is enabled.

TRACE_IO does not require application-layer initialization, but the code must include `log.h`. Use `LOG_IO(uint32_t tag, uint8_t num, uint8_t val, uint8_t flip)` to control the I/O. The parameters are defined as follows:

1. Tag: TRACE_IO condition. The interface compares this parameter with `CONFIG_LOG_IO_TAG_MASK` and performs the I/O operation only when the values match.
2. Num: Selects the I/O to control. For example, an input of 0 operates PA0.
3. Val: An input of 0 drives the I/O selected by Num low, and an input of 1 drives it high.

4. Flip: When nonzero, generates the specified number of pulses on the I/O selected by Num. Note the following:
 - When Flip is 0, Val controls whether the I/O is driven high or low.
 - When Flip is nonzero, Val is ignored. After generating the specified number of pulses, the I/O is driven low.

Before selecting a pin for TRACE_IO testing, see the RS6xxx Hardware Design and PCB Layout Guide.

5.5.2 Logging System

The SDK provides a set of logging APIs that provide more specific debug information than TRACE_IO. Before using the logging system, perform the following configuration in prj_config.h:

1. Define CONFIG_PRINTF, the master logging-system switch. Set it to 1 when logging is used.
2. Define CONFIG_PRINTF_UART_BAUDRATE to configure the logging baud rate. If undefined, it defaults to 115200. Modify it as required.
3. Define CONFIG_PRINTF_UART_NUM to select the UART used for logging. In UART-SPI/I²C mode, only UART0 can be selected for log output. In DUAL-UART mode, UART1 is recommended.

Note the following:

- The UART used by the logging system cannot be the same UART used by HIF.
4. **Define LOG_LEVEL before #include "log.h". This macro controls the logging output level. Logging APIs and the configured level control debug-information output. A higher configured level also enables APIs at lower levels. The values are as follows:**
 - 0: No information is output.
 - 1: Outputs errors that block functionality or indicate abnormal data.
 - 2: Outputs warnings that do not affect current functionality but require attention.
 - 3: Outputs normal operating information, such as status information.
 - 4: Outputs debug information and is recommended during development.
 5. **Define LOG_MODULE before #include "log.h". This macro must be a string and is included when a logging API is called. Define it based on the file name to help locate the source file from log information.**
 6. Include the log.h header file.

In the reference project, Steps 1 through 3 are defined in project/mmwave/common/radar_framework/inc/radar_framework_default_config.h.

When using the logging system, select an API based on the information type and use it with the log level defined in Step 4. The logging APIs are as follows:

1. LOG_PRINT: Used in the same way as the standard C-library printf function. Outputs log information when LOG_LEVEL is greater than 0.
2. LOG_ERR: Used in the same way as LOG_PRINT. Outputs log information when LOG_LEVEL is greater than or equal to LEVEL_ERR. It automatically prefixes the message with "[ERR "LOG_MODULE"]".
3. LOG_WRN: Similar to LOG_ERR. Outputs log information when LOG_LEVEL is greater than or equal to LEVEL_WRN and prefixes the message with "[WRN "LOG_MODULE"]".
4. LOG_INF: Similar to LOG_ERR. Outputs log information when LOG_LEVEL is greater than or equal to LEVEL_INF and prefixes the message with "[INF "LOG_MODULE"]".
5. LOG_DBG: Similar to LOG_ERR. Outputs log information when LOG_LEVEL is greater than or equal to LEVEL_DBG and prefixes the message with "[DBG "LOG_MODULE"]".

Note:

- LOG_MODULE in the automatically added prefix is the string defined by the LOG_MODULE macro. If the macro is not defined, the string is omitted.
- Calling an API above the configured log level suppresses the log output and does not cause a build error.

After completing the logging-system configuration, run the firmware and view LOG output with a serial debugging tool. Select the baud rate and port defined in Steps 2 and 3.

5.5.3 PSIC Debug Protocol

The SDK defines a subprotocol called the PSIC Debug Protocol within the HIF Payload field. It has the following key features:

1. It is a subprotocol within the HIF framework, uses the fixed HIF MsgID 0xC6, and places data in the HIF Payload, eliminating the need to develop a new application-layer protocol.
2. Supports reporting byte, short, word, float, double, and other data types, including arrays.
3. Supports reporting fixed-point bit widths and restoring them on the Host.
4. Supports different data organization methods.
5. Supports flexible data labels that identify each data type by a string.
6. Supports reporting up to 4090 bytes of data.
7. Supports string reporting and string display in RadarDebugTool.

The relationship between the PSIC Debug Protocol and the HIF Payload field is shown below:

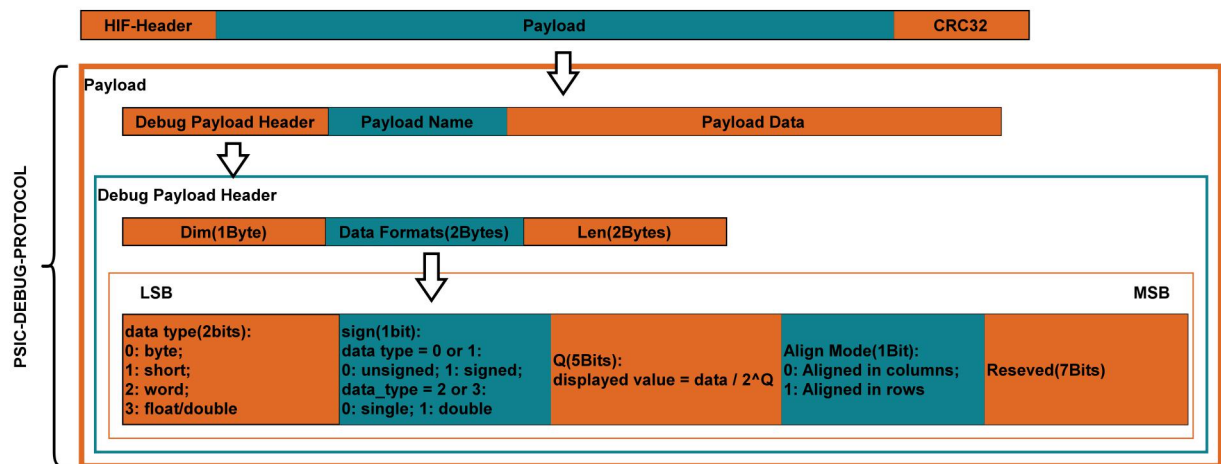


Figure 5-8 PSIC Debug Protocol

In the reference project, `radar_framework_report_debug_data()` reports data to the Host in the PSIC Debug Protocol format. This section describes the meaning of each debug parameter.

The Data Format field controls data organization. Its flags are defined as follows:

- Dim
 - Indicates the dimension of the data to report. For example, Cartesian coordinates x/y/z form three-dimensional data. RadarDebugTool supports visualization when Dim = 1, 2, or 3.
- Len
 - Indicates the number of multidimensional data items to report. $\text{Dim} \times \text{Len}$ is the total number of reported elements.
- Align Mode (Data Alignment)
 - When Align Mode = 0, Payload Data is filled first with Len values of the same dimension. After one dimension is complete, Len values of the next dimension are filled until all Dim dimensions are complete.

For example, when filling N x/y/z Cartesian coordinates with Dim = 3 and Align Mode = 0, the PSIC Debug Protocol data is arranged as follows:

x1	x2	...	xN	y1	y2	...	yN	z1	z2	...	zN
----	----	-----	----	----	----	-----	----	----	----	-----	----

- When Align Mode = 1, Payload Data is filled first with Dim values from different dimensions, followed by the next group of Dim values, until all Len multidimensional items are complete.

For example, when filling N x/y/z Cartesian coordinates with Dim = 3 and Align Mode = 1, the PSIC Debug Protocol data is arranged as follows:

x1	y1	z1	x2	y2	z2	...	xN	yN	zN
----	----	----	----	----	----	-----	----	----	----

- Q (Fixed-Point Bit Width)

- After parsing Payload Data, RadarDebugTool divides the data by 2^Q for display and storage. This is useful for fixed-point reported data. For floating-point data, Q is generally set to 0.
- Data Type
 - Indicates the data type of each element. Supported types include byte, short, word, float, and double.
- Sign (Sign Flag/Floating-Point Type)
 - When Data Type is byte, short, or word, Sign = 1 indicates that all values in Data Payload are unsigned. Sign = 0 indicates that all values in Data Payload are treated as unsigned by the Host.
 - When Data Type is float/double, Sign = 1 indicates double, and Sign = 0 indicates float.
- Payload Name
 - Payload Name identifies the label of the debug data. It is a string that must end with \0. This label determines the name used to display and store the data in RadarDebugTool.
- Payload Data
 - Fill the data according to Align Mode, Dim, and Len.

For example, frame_info reporting in the Tracking project is displayed in RadarDebugTool as shown in Figure 5-9. The LOG workspace on the right displays the received raw hexadecimal data. The following is an example bitstream received for one frame:

```
A5 2C 32 C6 26 10 01 01 00 0B 00 66 72 61 6D 65 5F 69 6E 66 6F 00 60 00 04 00 F4 01 00 19 00 01 40 06
20 00 00 00 00 00 00 00 00 00 00 76 03 54 FA
```

Bitstream breakdown:

C6: MSG_ID in the HIF Header shown in Figure 5-8. All data reported using the PSIC Debug Protocol uses C6.

01 01 00 0B 00: Corresponds to Debug Payload Header in Figure 5-8.

- 0x01 indicates that Dim is 1.
- 0x01 indicates the following:
 - Data Type is 01 (2 bits), indicating short.
 - Sign is 0 (1 bit), indicating unsigned data.
 - Q is 00000 (5 bits), indicating 0.

■ 0x00 indicates that Align Mode is 0.

■ 0x0B and 0x00 form Len. The low byte comes first, giving 0x000B and a data length of 11.

66 72 61 6D 65 5F 69 6E 66 6F 00: Corresponds to Payload Name in Figure 5-8 and represents the ASCII string frame_info.

60 00 04 00 F4 01 00 19 00 01 40 06 20 00 00 00 07 00 00 00 00 00: Corresponds to Payload Data in Figure 5-8. It uses little-endian byte order, and the data length and other information correspond to the Debug Payload Header above.

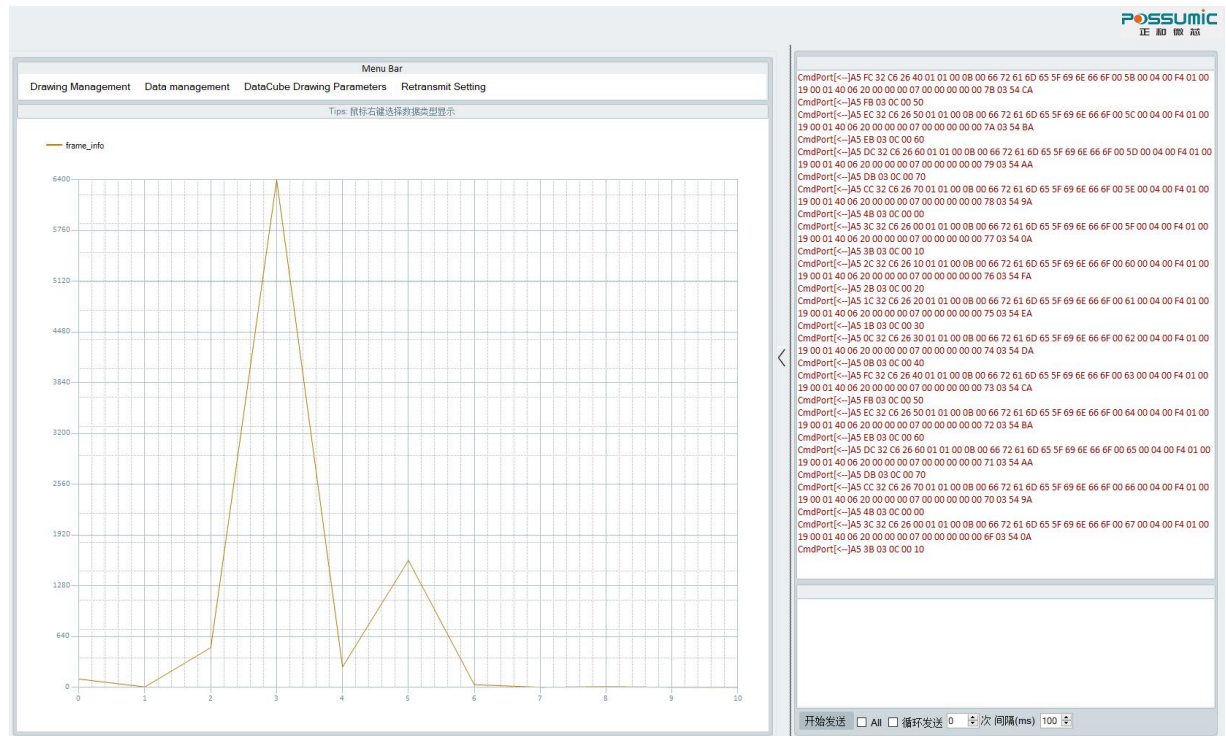


Figure 5-9 frame_info Display in the Tool

5.6 Reliability Design

The mmWave example project uses four debugging mechanisms to verify that the radar is operating correctly. Two mechanisms require detection on the Host.

5.6.1 Watchdog

Use CONFIG_MMW_WATCHDOG to enable the system watchdog. When the watchdog is enabled, CONFIG_WDG_TIMEOUT_MS sets the watchdog reset timeout, which defaults to 5 seconds.

Use the following interfaces to control the watchdog:

- watchdog_init(): Creates and initializes the watchdog device and starts the watchdog.
- watchdog_stop(): Stops the watchdog.
- watchdog_open(): Starts the watchdog after watchdog_stop() has been called.
- mmw_watchdog_feed(): Feeds the watchdog.

5.6.2 Task Monitor

Use Task Monitor to verify that radar frame-control processing in mmw_ctrl is operating correctly.

- mmw_loop_task_time_init(): Creates and starts a Task Monitor.
- mmw_loop_task_monitor_update(): Optimizes the Task Monitor interval according to the frame period.
- mmw_loop_task_time_reset(): Call this function in each callback to reset Task Monitor. Call it after Task Monitor is stopped to restart Task Monitor.
- mmw_loop_task_time_stop(): Stops Task Monitor.

5.6.3 Heartbeat Packet

The SDK supports periodic heartbeat reporting through an HIF message with MsgId = 0xFE. When an application does not periodically report targets, the Host can use heartbeat packets to determine whether the radar is still active.

A typical heartbeat HIF bitstream in TLV format is:

```
A5 D6 32 FE 14 40 00 04 00 00 00 00 01 04 8B 0E 00 00 02 02 00 00 03 02 00 00 ED EB E9 BB
```

The bitstream is interpreted as follows:

32 FE 14 40 is the HIF Header. MsgID = 0xFE and the length is 0x014 bytes.

00 04 00 00 00 00 is one TLV item:

Tag is 1 byte: 00 indicates RESPONSE_CNT.

Length is 1 byte: 04 indicates that the following Value is 4 bytes.

Value is 0.

Heartbeat packet Tags are defined as follows:

RESPONSE_CNT = 0: Value is the number of HIF Response messages sent.

FRAME_INDEX = 1: Value is the frame index, starting from 1.

MOTION_PT_NUM = 2: Value is the motion point-cloud count for the current frame.

PRESENT_PT_NUM = 3: Value is the presence point-cloud count for the current frame.

For the HIF message format, see the RS6x_7x HIF Development Quick Start Guide.

5.6.4 Response Count

Each Host command produces one Device Response, after which Response Count increments. The Host can periodically obtain Response Count through an HIF Command and compare it with the previous value. If the value decreases, cannot be obtained, or communication is abnormal, the radar is considered abnormal and the Host can restart it.

The HIF message used to obtain Response Count has MsgId = 0x01 and SubId = 0x41. The returned HIF Payload contains a 1-byte Status and a 4-byte Response Count.

Example bitstream:

Host --> Radar:

A5 43 15 01 01 00 41 A9 FE FE FF

Radar --> Host:

A5 42 **12 01 05 00** **00 01 00 00 00** ED FD FA FF

12 01 05 00: HIF Header

00 01 00 00 00: status = 0 indicates successful reception. Response Count = 0x00000001 indicates that one Response had already been sent before this Command was received.

5.7 Embedded Resource Configuration and Verification

5.7.1 Configuring Code Resources

During user-code development, code runs from XIP by default. The following two methods place code in SRAM:

1. Add the wildcard for the corresponding file to isram_text and isram_rodata in the link file:

```
isram_text : {
    . = ALIGN(0x4) ;
    __s_isram_text = . ;

    /* The asterisks before and after add_new_file are file wildcards. The leading asterisk is strongly
    recommended; the trailing asterisk is optional. */
    *add_new_file*.o(*.text*)

    . = ALIGN(0x4) ;
    __e_isram_text = . ;
} > ISRAM
```

2. Add the `__sram_text` or `__keep_sram_text` qualifier before a function definition to link it to SRAM, for example: `__sram_text void my_func();`

The link file is located at the path specified by the project configuration:

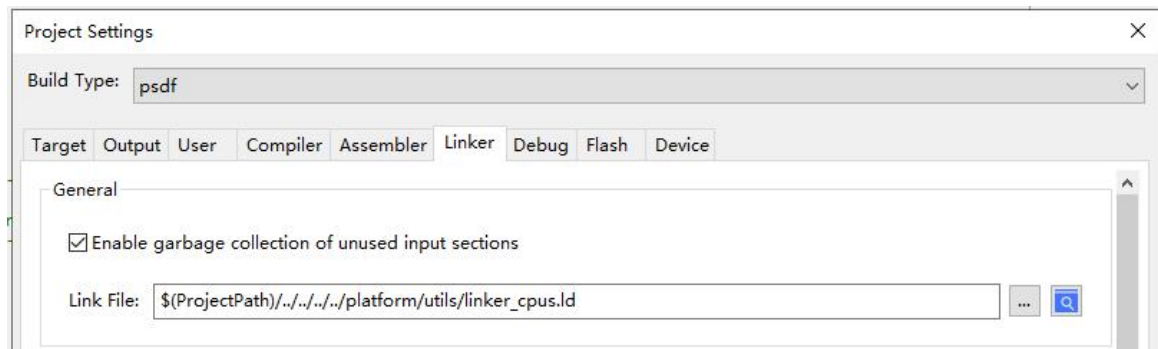


Figure 5-10 Link-File Path

Functions qualified with `__sram_text` are built regardless of whether the defined functions or variables are used, which may waste resources. Use either of the following methods to avoid this:

1. For a specific function, use a macro to control whether the function is enabled, and use the same macro to control the definition and use of its functions or variables so that disabled functions do not consume resources.
2. If code is not called by another module and must be placed in SRAM with `__sram_text`, qualify it with `static`. An unused function is then omitted from the build, avoiding wasted resources.

Section Headers:

[Nr]	Name	Type	Addr	Off	Size	ES	Flg	Lk	Inf	AI
[0]		NULL	00000000	000000	000000	00		0	0	0
[1]	head_pad	PROGBITS	09000000	011000	000020	00	WA	0	0	1
[2]	isram_text	PROGBITS	00500000	002000	00da2c	00	WAX	0	0	64
[3]	isram_rodata	PROGBITS	0050da30	00fa30	000c48	00	A	0	0	8
[4]	ipsram_text	PROGBITS	0d000000	023654	000000	00	W	0	0	1
[5]	ixip_text	PROGBITS	09000020	011020	00fb94	00	AX	0	0	2
[6]	ixip_rodata	PROGBITS	0900fbb8	020bb8	0023f0	00	A	0	0	8
[7]	dsram_data	PROGBITS	10500000	023000	000654	00	WA	0	0	8
[8]	dsram_bss	NOBITS	10500658	023654	000a98	00	WA	0	0	8
[9]	dsram_stack	NOBITS	105010f0	023654	00c800	00	WA	0	0	16
[10]	ipsram_rodata	PROGBITS	0d000000	023654	000000	00	W	0	0	1
[11]	dpsram_data	PROGBITS	1d100000	023654	000000	00	W	0	0	1
[12]	dpsram_bss	PROGBITS	1d100000	023654	000000	00	W	0	0	1
[13]	.info	PROGBITS	00000000	001000	0000ce	00	A	0	0	4
[14]	.comment	PROGBITS	00000000	023654	000093	01	MS	0	0	1
[15]	.riscv.attributes	RISCV_ATTRIBUTE	00000000	0236e7	000125	00		0	0	1
[16]	.debug_info	PROGBITS	00000000	02380c	0040e4	00		0	0	1
[17]	.debug_abbrev	PROGBITS	00000000	0278f0	0018c6	00		0	0	1
[18]	.debug_loclists	PROGBITS	00000000	0291b6	004dd6	00		0	0	1
[19]	.debug_aranges	PROGBITS	00000000	02df8c	000278	00		0	0	1
[20]	.debug_line	PROGBITS	00000000	02e204	009295	00		0	0	1
[21]	.debug_str	PROGBITS	00000000	037499	0008c1	01	MS	0	0	1
[22]	.debug_line_str	PROGBITS	00000000	037d5a	000406	01	MS	0	0	1
[23]	.debug_frame	PROGBITS	00000000	038160	00034c	00		0	0	4
[24]	.debug_rnglists	PROGBITS	00000000	0384ac	0007b6	00		0	0	1
[25]	.symtab	SYMTAB	00000000	038c64	0091e0	10		26	1236	4
[26]	.strtab	STRTAB	00000000	041e44	0070f6	00		0	0	1
[27]	.shstrtab	STRTAB	00000000	048f3a	000142	00		0	0	1

Figure 5-11 Memory Distribution in the MAP File

For more information about configuring code execution locations, see the '[RS6x_7x_SDK_Reference_Manual](#)'.

5.7.2 Verifying Embedded Resources

The preceding section described how to configure code resources during development and how to determine the resource usage of a specific function. This section describes overall resource statistics.

- (3) Place psdf.asm and [/Software Kit/02_SDK/psdf_sdk/tools/scripts/find_xip_refs](#) in the same directory.
 - (4) Open CMD or another command-line tool in this directory and run `python find_xip_refs.py --asm psdf.asm radar_framework_2d_frame_cb`. Here `radar_framework_2d_frame_cb` is the function to test and can be changed as required.
 - (5) After the script completes, it displays functions that execute from XIP and saves the results in a .csv file with the same name as the interface. The .csv file can be used for detailed inspection. If the queried function is not present in psdf.asm, the script reports that the function was not found in the file.
2. Alternatively, open psdf.map in the Lst subdirectory under the built project, identify the SRAM address range shown in Figure 5-11, and search for the function. Determine whether its address is in SRAM. For example, `radar_framework_2d_frame_cb` is located at 0x0050d866, while the SRAM range is 0x00500000 to 0x0050da2c, so the function is in SRAM.

5.7.3 Measuring Computation Time

This section describes two methods provided by the SDK for measuring code computation time:

1. Use TRACE_IO as described in Section 5.5.1. Drive the selected I/O high before the code under test and low afterward. The high-level duration measured with a logic analyzer or oscilloscope is the computation time. PA0 example:

```
#include "log.h"
#define CONFIG_LOG_IO_TAG_MASK      (0x1000)
#define CONFIG_LOG_IO_MASK         (0x01) /* Enable PA0 for LOG */

LOG_IO(CONFIG_LOG_IO_TAG_MASK, 0, 1, 0); /* Drive PA0 high */
/* Execute the code under test */
LOG_IO(CONFIG_LOG_IO_TAG_MASK, 0, 0, 0); /* Drive PA0 low */
```

2. Use the SDK HAL API to measure computation time. After including `hal_board.h`, call `HAL_BOARD_GetTime(HAL_TimeType_t timeType)` to obtain the current time. `timeType` selects the unit. The following values are supported:
 - `HAL_TIME_CYCLE`: Returns the current time in CPU cycles.
 - `HAL_TIME_US`: Returns the current time in microseconds.
 - `HAL_TIME_MS`: Returns the current time in milliseconds.
 - `HAL_TIME_S`: Returns the current time in seconds.

Use the timestamp difference as the computation time and report it through the logging system or debug protocol:

```
#define LOG_LEVEL                    1
#define LOG_MODULE                   "my_module"
#include "log.h"
#include "hal_board.h"
uint32_t time_start = HAL_BOARD_GetTime(HAL_TIME_US);
/* Execute the code under test */
uint32_t time_end = HAL_BOARD_GetTime(HAL_TIME_US);
LOG_PRINT("Calculate the required time:%d(us)\n", time_start - time_end);
```

5.8 Porting and Verifying Low-Power Functions

5.8.1 Configuring Low-Power Mode

Set `CONFIG_PM` to 1 to enable sleep/wake mode. After the radar completes its computation tasks, the system idle thread places the Device in a low-power state. The radar enters sleep after completing frame transmission. See Figure 5-14 for the configuration. For details, see the '[RS6x_7x_SDK_Reference_Manual](#)'.

```
/* power mange config
 */
/**
 * @brief If enable 'CONFIG_PM', after CPU finished processm, CPU will shut down and consumes a current of 10uA between frames.
 */
#ifndef CONFIG_PM
#define CONFIG_PM 0
#endif
#define CONFIG_PMU_EXTLDO_SLEEP_SW_SUBMODE 1
```

Figure 5-14 Low-Power and CPU-Performance Configuration

When low-power mode is used, configure and use WKIO. During system sleep, peripherals are powered off and standard I/Os cannot retain their output levels or receive a wake-up signal from the controller. WKIO can retain specified output levels and input interrupts during sleep. The WKIO example is located at [psdf_sdk/project/driver/wkio](#).

5.8.2 Verifying Power Consumption

Prepare an ammeter and perform the following procedure to measure power consumption:

1. Disconnect VDD3V3-VCC at the lower right of Figure 2-1 and connect VDD13-V13. For the MRS6130-AIP-DEV-V1 or MRSx241-DEV-V1 development board, only disconnect VDD3V3-VCC.
2. Point the hardware toward an open area to reduce point-cloud computation time and allow the hardware to enter sleep more quickly.
3. Connect the positive power-supply terminal to VCC on the baseboard and the negative terminal to GND. Reversed polarity may damage the hardware when power is applied. Verify the connections before turning on the supply.
4. Turn on the power supply and use its software to observe current. For the example project, the current waveform for one frame should show radar transmission, CPU computation, HIF data transfer, and sleep when PM is enabled. Measure the following:
 - Average power consumption: Accumulate at least 2 minutes of current waveforms and record the average current or power.
 - Sleep current: With PM enabled, record the average sleep-state current. In theory, the sleep quiescent current does not exceed 10 μ A.

Note the following:

1. When PM is enabled, SPI communication is used, and HIF data-transfer operations are present, connect the Host tool and start scanning during power testing. Otherwise, HIF affects the time required to enter sleep.
2. Depending on power-supply performance, the quiescent current may continue increasing after sleep begins, with longer sleep producing a higher quiescent current. For a more accurate measurement, increase the frame period so that the sleep duration exceeds 500 ms.
3. During power testing, set the VCC-5V single-pole double-throw switch beside POWER in Figure 4-1 to VDD3V3; otherwise, the measured current is inaccurate.

5.9 Other SDK Modifications

In addition to communication-interface changes, porting to target hardware may require other detailed modifications. The SDK provides example programs for these functions. Refer to the examples to quickly configure system functions.

- **OTA: Two OTA approaches are available: BURN upgrade and OTA upgrade. BURN upgrade uses Burn to operate directly on the radar Flash area. OTA upgrade burns firmware to a backup Flash area and swaps the old and new IMG files after system restart. The key difference is that BURN upgrade requires Flash capacity equal to only one maximum IMG size, while OTA upgrade requires at least twice the IMG size. For more information, see the RS6x_7x Firmware Download Development Manual.**

- **CPU performance configuration:** Add `CONFIG_SOC_CLOCK_CFG` to `prj.config.h` to change the preset CPU clock, as shown in Figure 5-14. Preset clock frequencies are listed in Table 5-4.

Table 5-4 Default CPU Frequencies

	CONFIG_SOC_CLOCK_CFG Value	CPU Operating Frequency
RS6130-P1806	Not Specified	128 MHz
RS6130-P1812	Not Specified	128 MHz
RS6240	Not Specified	256 MHz
RS6241-P2828	Not Specified	256 MHz
RS6241-P2840	Not Specified	256 MHz

For available CPU clocks, see [psdf_sdk/platform/soc/rs613x/rs6130_config.h](#), [psdf_sdk/platform/soc/rs624x/rs6240_config.h](#), and the '[RS6x_7x_SDK_Reference_Manual](#)'.

6 Radar Debugging Methods

This chapter describes how to use RadarDebugTool effectively in the development flow shown in Figure 1-1. It addresses the high data throughput and limited visualization that can make radar-system debugging difficult, helping developers quickly locate and analyze issues. During design and evaluation, RadarAnalysisTool helps verify waveform configurations and design algorithms. For evaluation methods, see the RS6x_7x mmWave Radar Evaluation Quick Start Guide.

During development, RadarDebugTool provides the following functions:

1. Efficient DataCube acquisition and visualization: With a single-line SPI hardware interface, reports up to a 256 KB DataCube at 10 Hz and provides multiple DataCube visualizations for troubleshooting.
2. Multiple debug-information visualization methods: Supports custom debug-information reporting, visualization, and storage.

For more information about RadarDebugTool, see the RS6x_7x RadarDebugTool User Manual.

6.1 Generating Radar Configuration Interfaces

After confirming the waveform configuration and algorithm prototype with RadarAnalysisTool during design and evaluation, developers can generate a hardware-configuration function-call example with one click and import it into the project.

6.2 Testing and Verification

During testing and verification, use the PSIC Debug Protocol described in Section 5.5.3 to visualize arbitrary data and monitor custom software status in real time using charts. RadarDebugTool real-time monitoring and DataCube storage help identify abnormal data during testing and locate system bugs.

For information about RadarDebugTool data visualization, see the RS6x_7x RadarDebugTool User Manual and the RS6x_7x mmWave Radar Sample Reference Manual.

7 Q & A

Q: During initial development and verification on the development board, should UART-SPI mode or DUAL-UART mode be selected?

A: Because radar debug data has a high data volume, UART-SPI mode is recommended for development-board use. It reduces transfer time and supports a higher radar frame rate.

Q: Why does the radar LOG UART print "mmw_process_fft_ant_f32 call failed, err_code: 4"?

A: This error occurs because the interval between completion of radar transmission and the next Frame is too short. The hardware writes data to the DataCube during transmission while software is still accessing the DataCube, causing a conflict and potentially corrupting the current processing flow. Limit the maximum point count or reduce the frame rate to avoid the issue.

Q: Why does the build report that xxx.o cannot be found and then fail?

A: When the code directory structure changes, residual link information may not be removed completely. Click Clean Project, or manually delete the Lst and Obj directories under the project directory, and rebuild the project.

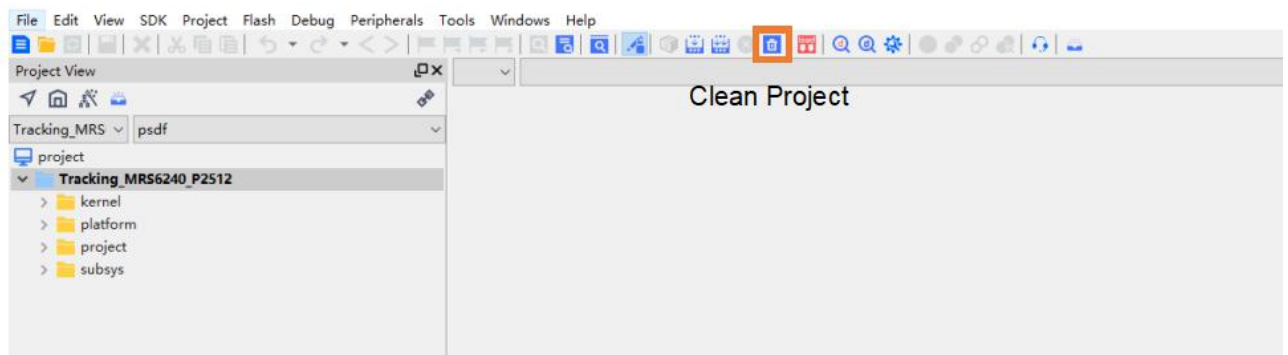


Figure 7-1 Clean Project

Q: How can the radar be connected to other embedded hardware for development?

A: Use the header holes on both sides of the development board. UART0 corresponds to PB0-TX and PB1-RX; UART1 corresponds to PA0-TX and PA1-RX; SPI corresponds to PA0-CS, PA1-SCLK, PA2-MOSI, and PA3-MISO; and I²C corresponds to PA4-SDA and PA5-SCL.

Before connecting, verify that the resistors between the module pins and development-board hardware are soldered correctly. The resistors discussed here are highlighted by the green box in the following figure and are numbered in pairs.

The MRS6240-AIP-DEV-V1 has six resistor groups, ordered from left to right as 0A, 0B, 1A, 1B, ..., 4A, 4B, 5B, 5A. Group 0 corresponds to PA5, Group 1 to PA4, ..., and Group 5 to PA0. A connects to the development-board header hole and B connects to FUNC SEL or the bridge-device pin. 0B and 2B connect directly to the bridge device.

The MRS6130-AIP-DEV-V1 has seven resistor groups, ordered from top to bottom as 0A, 0B, 1A, 1B, ..., 6A, 6B. Group 0 corresponds to PA5, Group 1 to PA4, ..., and Group 5 to PA0. Group 6 is special and is described below. A connects to the development-board header hole and B connects to FUNC SEL or the bridge-device pin. 0B and 2B connect directly to the bridge device. 6A and 6B connect to the POUT pin (Pin 2) of the MRS6130-P1806 module. 6A connects POUT to PA0 on the development board, and 6B connects POUT to PA6.

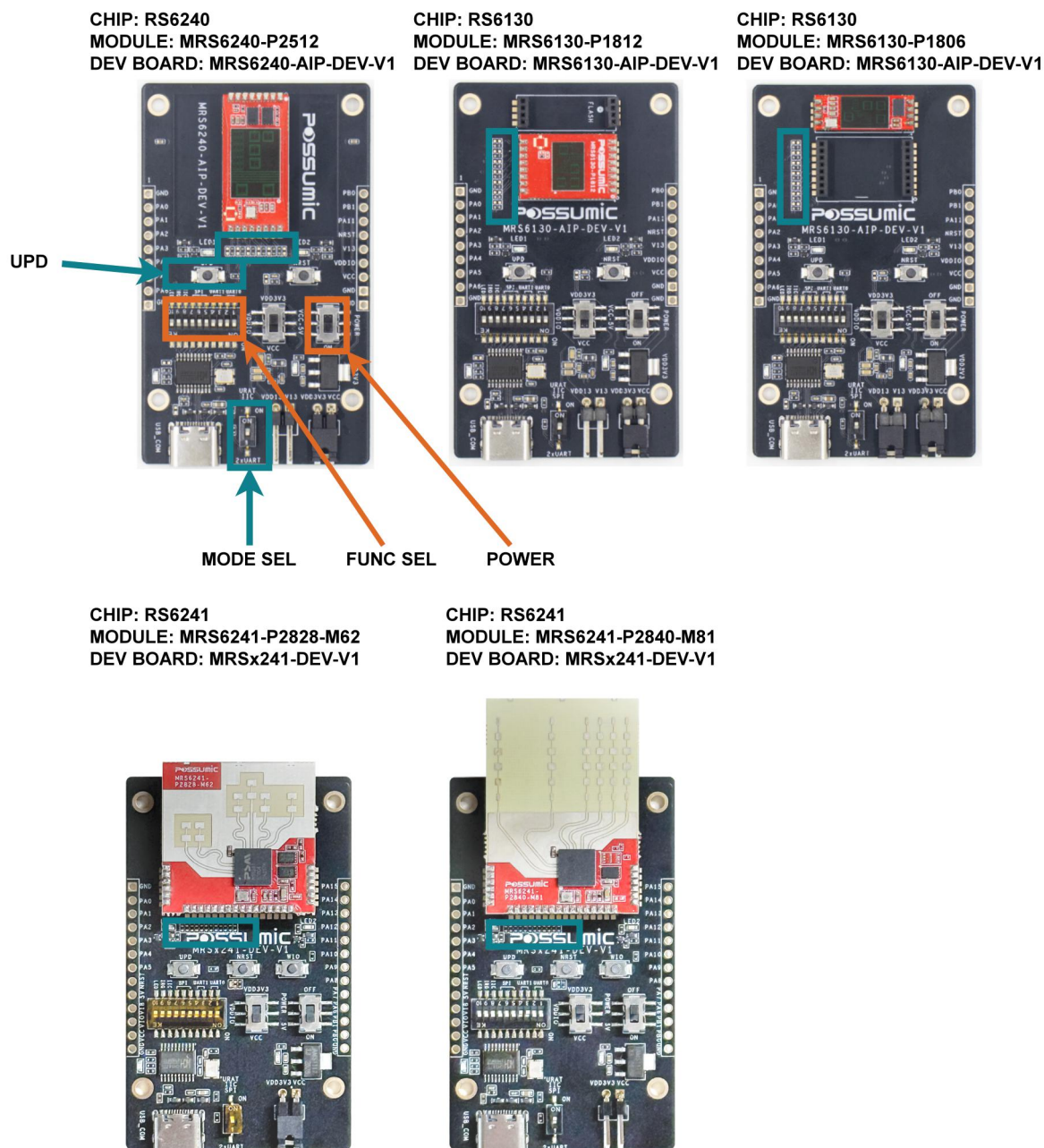


Figure 7-2 Resistors Connecting Module Pins to Development-Board Hardware

For information about developing a radar Host, see the RX6x_7x HIF Reference Manual.

Disclaimer and Copyright Notice

The information provided by our company in this document is accurate to the best of our knowledge, but we do not guarantee that the content is completely error-free. All liability for any damages arising from the use of this document (including but not limited to indirect, incidental, or special losses) or any improper use resulting from non-compliance with the instructions herein (including but not limited to over-voltage, over-frequency, or over-temperature operations) is explicitly disclaimed.

The content in this document, including URL references, may differ from the actual content due to product version upgrades or other reasons, and is subject to change without notice.

THIS DOCUMENT IS PROVIDED "AS IS" WITH NO WARRANTIES. NO WARRANTY IS PROVIDED TO THIS DOCUMENT FOR ITS MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, NOR DOES ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

All liability, including liability for infringement of any patent rights, relating to use of information in this document is disclaimed. No licenses express or implied, by estoppel or otherwise, to any intellectual property rights are granted herein.

Copyright © 2026 Possumic Technology Co., Ltd. All rights reserved.